EXPERIMENT 21

Containerize a Python Flask Application Using Docker

1. Aim

To develop a simple Python Flask To-Do List application, containerize it using Docker, run it locally, test its functionality, and push the Docker image to Docker Hub.

2. Requirements

Python 3, Flask, Docker, Web Browser, and Docker Hub.

3. Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

4. Important Docker Commands

```
docker build -t todo-flask:latest .
docker run -d --name todo-flask -p 5000:5000 todo-flask:latest

docker login
docker tag todo-flask:latest <username>/todo-flask:latest
docker push <username>/todo-flask:latest
```

5. Testing

Open localhost:5000 in the browser. The To-Do List application should load and allow tasks to be added, completed/undone, and deleted.

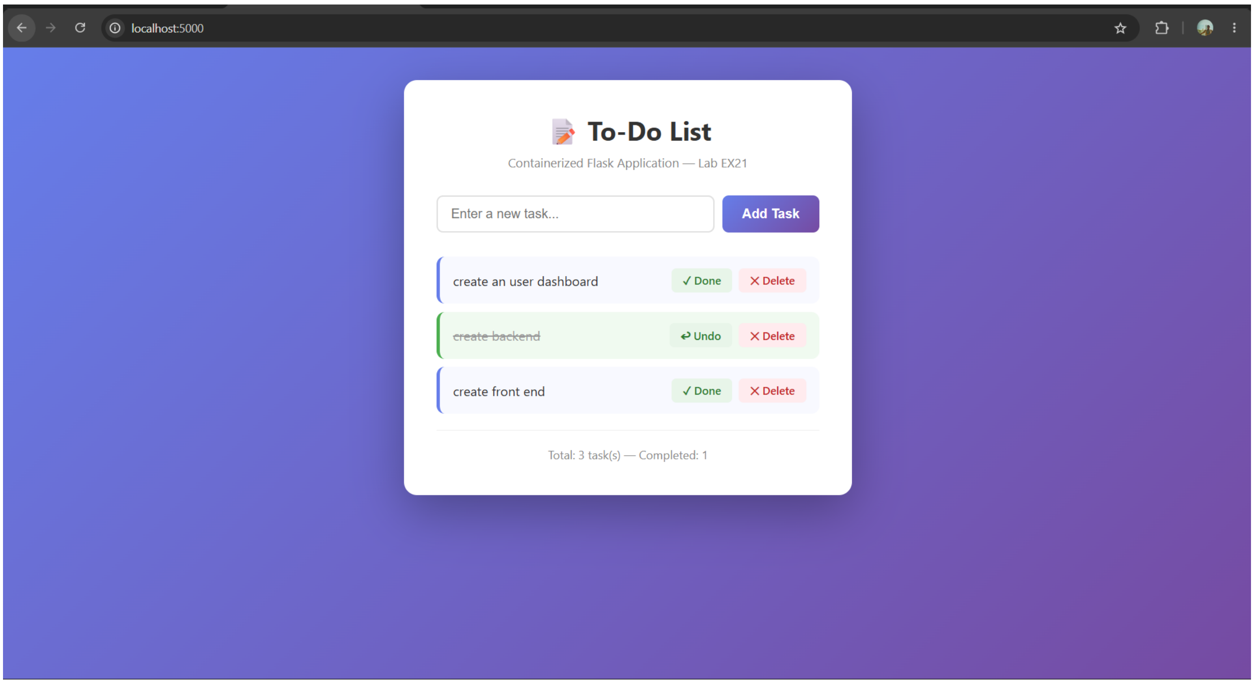

Figure 1: Testing result of the Flask To-Do List application running on localhost:5000.

6. Result

The Flask To-Do List application was successfully containerized and tested inside a Docker container. The application was accessible through the browser, and the Docker image was prepared for publishing to Docker Hub.

7. Conclusion

Docker provides an isolated and reproducible environment for running the Flask application and simplifies deployment through Docker images and Docker Hub.